

iOS工程接入

大约 15 分钟 约 4380 字

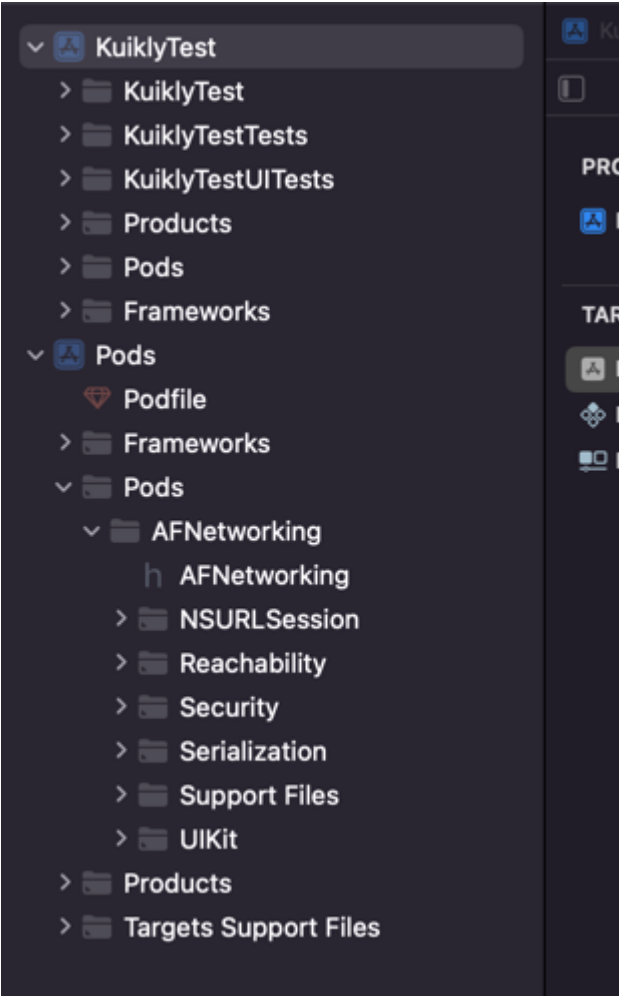
注意

在此之前请确保已经完成**KMP侧 Kuikly**的接入，如还未完成，请移步[KMP跨端工程接入](#)

完成**Kuikly KMP**侧的配置后, 我们还需要将**Kuikly**渲染器和适配器接入到宿主平台中，此文档适用于您想把Kuikly渲染器接入到您现有的iOS工程中。下面我们来看下，如何在现有iOS工程中接入Kuikly渲染器。

我们先新建一个名为**KuiklyTest**新工程并假设这个工程是你现有的iOS工程

反馈文档建议



添加Kuikly iOS 渲染器依赖

方式一：通过 CocoaPods 集成（推荐）

1. 添加 kuikly ios render , 在你的工程的podfile添加以下代码

```
source 'https://cdn.cocoapods.org/'

platform :ios, '14.1'

target 'KuiklyTest' do

  inhibit_all_warnings!

  pod 'OpenKuiklyIOSRender', 'KUIKLY_RENDER_VERSION'
end
```

rb

反馈
文档
建议

- KUIKLY_RENDER_VERSION 需要替换为实际的 Kuikly 版本号，在这里[查看最新版本](#)
- 版本号需要和[KMP跨端工程](#)保持一致

2. 执行 `pod install --repo-update` 安装依赖

方式二：通过 SPM（Swift Package Manager）集成

Kuikly iOS 渲染器已支持通过 SPM 集成，推荐 Xcode 12 及以上版本使用。

1. 添加 Kuikly SPM 依赖

- 打开 Xcode，选择你的项目工程，点击左侧导航栏的 **Project**。
- 选择 **Package Dependencies** 标签页，点击右下角的 + 按钮。

- 在弹出的对话框中，输入 Kuikly iOS 渲染器的 Git 仓库地址：

```
https://github.com/Tencent-TDS/KuiklyUI.git
```

sh

- 选择你需要的版本（建议与 KMP 工程保持一致），点击 **Add Package**。
- 在弹出的选择框中，勾选你的 Target，点击 **Add Package** 完成依赖添加。

提示

如需指定版本号，请选择与 KMP 工程一致的版本号。

2. 链接 Kuikly 业务代码 framework

Kuikly 业务代码在 iOS 平台会被编译为 `.xcframework`，推荐以下集成方式：

- 推荐：通过 SPM 集成业务 `.xcframework`**

建议将业务 `shared.xcframework` 封装为一个本地或私有的 Swift Package，然后通过 SPM 集成到主工程。

参考步骤如下：

- 新建一个 Swift Package（如 `shared`），并将 `shared.xcframework` 拷贝到该包目录下。
- 在 `Package.swift` 中添加：

```
.binaryTarget(  
    name: "shared",  
    path: "./shared.xcframework"  
)
```

swift

- 在目标工程中通过 SPM 添加该 Package 依赖。
- 对于图片等资源文件，由于 Kuikly 默认从 `mainBundle` 内按路径加载，因此有如下几种处理方式：
 - 方式一：**将图片资源文件直接拖动至工程中，由 Xcode 自动在打包时拷贝到 `main bundle` 内，注意需对齐 Kuikly 侧的使用方式，按需保留文件夹结构。

- **方式二：**使用SPM管理资源，需如下两步：

a. 准备资源package：修改上述或新建 `Package.swift`，添加资源target，如下所示：

```
.target(swift
    name: "SharedResource",
    resources: [
        // 注意使用.copy命令而非.process，以保留文件夹结构
        .copy("KuiklyResources") // KuiklyResources 为资源目录名，可任意命名
    ],
),
```

将Kuikly项目所需的资源文件拷贝至上述KuiklyResources目录下，并适当增加include等必要目录结构以符合Swift Package要求。

最终完整 `Package.swift` 应与如下示例类似：

```
// swift-tools-version: 5.7swift
import PackageDescription

let package = Package(
    name: "shared",
    platforms: [.iOS(.v13)],
    products: [
        .library( name: "shared", targets: ["shared"]),
        .library(name: "SharedResource", targets: ["SharedResource"])
    ],
    targets: [
        .binaryTarget(
            name: "shared",
            path: "Frameworks/shared.xcframework",
        ),
        .target(
            name: "SharedResource",
            resources: [
                .copy("KuiklyResources")
            ],
        ),
    ],
)
```

b. 修改工程接入设置，新增如下代理回调方法，返回资源加载目录：

```
// MARK: - KuiklyViewBaseDelegate
func resourceFolderUrl(forKuikly pageName: String) -> URL {
    return
    Bundle.main.bundleURL.appendingPathComponent("shared_SharedResource.bundle/K
uiklyResources")
}
```

- **方式三：**自定义资源Swift Package, 并参考下文 [实现图片加载适配器](#) 一节进行自定义适配，注意确保路径正确匹配。

- **其他方式：**

- **手动集成：**将 `shared.xcframework` 拖入 Xcode 工程，并设置为 `Embed & Sign`。

3. 其他说明

- 适配器实现、Kuikly 容器等代码与 Pod 方式一致，参考下文实现即可。
- SPM 方式下，部分三方库（如 SDWebImage）如需使用，请自行通过 SPM 或 CocoaPods 集成。

反馈
文档
建议

实现Kuikly承载容器

以 UIViewController 方式接入

在Kuikly iOS工程中，默认是采用UIViewController 作为端侧承载 KuiKly Page的容器。可通过新建KuiklyRenderViewController, 构建 kuikly 页面的容器。

具体实现代码，请参考源码工程iOSApp模块的 KuiklyRenderViewController 类。

```
#import <UIKit/UIKit.h>
NS_ASSUME_NONNULL_BEGIN

@interface KuiklyRenderViewController : UIViewController

/*
 * @brief 创建实例对应的初始化方法.
 * @param pageName 页面名 （对应的值为kotlin侧页面注解 @Page("xxxx") 中的xxx名）
 * @param params 页面对应的参数（kotlin侧可通过pageData.params获取）
 * @return 返回KuiklyRenderViewController实例
 */
```

```

    */
- (instancetype)initWithPageName:(NSString *)pageName pageData:(NSDictionary
*)pageData;
@end

NS_ASSUME_NONNULL_END

```

```

#import "KuiklyRenderViewController.h"
#import <OpenKuiklyIOSRender/KuiklyRenderViewControllerBaseDelegator.h>
#import <OpenKuiklyIOSRender/KuiklyRenderContextProtocol.h>

#define HRWeakSelf __weak typeof(self) weakSelf = self;
@interface KuiklyRenderViewController()
<KuiklyRenderViewControllerBaseDelegatorDelegate>

@property (nonatomic, strong) KuiklyRenderViewControllerBaseDelegator *delegator;

@end

```

反馈文档建议

```

@implementation KuiklyRenderViewController {
    NSDictionary *_pageData;

- (instancetype)initWithPageName:(NSString *)pageName pageData:(NSDictionary
*)pageData {
    if (self = [super init]) {
        // 存储页面数据
        pageData = [self p_mergeExtParamsWithOriditalParam:pageData];
        _pageData = pageData;

        // 实例化Kuikly委托者类
        // ⚠️ 注意: 必须使用 KuiklyRenderViewControllerDelegator
        // 不要使用 KuiklyRenderViewControllerBaseDelegator
        _delegator = [[KuiklyRenderViewControllerDelegator alloc]
initWithPageName:pageName pageData:pageData];

        _delegator.delegate = self;
    }
    return self;
}

- (void)viewDidLoad {
    [super viewDidLoad];
}

```

```

self.view.backgroundColor = [UIColor whiteColor];
// 通知Kuikly页面ViewDidLoad
[_delegator viewDidLoadWithView:self.view];
[self.navigationController setNavigationBarHidden:YES animated:NO];
}

```

```

- (void)viewDidLayoutSubviews {
    [super viewDidLayoutSubviews];
    // 通知Kuikly页面viewDidLayoutSubviews
    [_delegator viewDidLayoutSubviews];
}

```

```

- (void)viewWillAppear:(BOOL)animated {
    [super viewWillAppear:animated];
    // 通知Kuikly页面viewWillAppear
    [_delegator viewWillAppear];
    [self.navigationController setNavigationBarHidden:YES animated:NO];
}

```

反馈文档建议

```

- (void)viewDidAppear:(BOOL)animated {
    [super viewDidAppear:animated];
    // 通知Kuikly页面viewDidAppear
    [_delegator viewDidAppear];
    [self.navigationController setNavigationBarHidden:YES animated:NO];
}

```

```

- (void)viewWillDisappear:(BOOL)animated {
    [super viewWillDisappear:animated];
    // 通知Kuikly页面viewWillDisappear
    [_delegator viewWillDisappear];
}

```

```

- (void)viewDidDisappear:(BOOL)animated {
    [super viewDidDisappear:animated];
    // 通知Kuikly页面viewDidDisappear
    [_delegator viewDidDisappear];
}

```

```

#pragma mark - private

```

```

- (NSDictionary *)p_mergeExtParamsWithOriditalParam:(NSDictionary *)pageParam {
    NSMutableDictionary *mParam = [(pageParam ?: @{}) mutableCopy];
}

```

```

        return mParam;
    }

#pragma mark - KuiklyRenderViewControllerDelegateDelegate

// 创建等待加载视图
- (UIView *)createLoadingView {
    UIView *loadingView = [[UIView alloc] init];
    loadingView.backgroundColor = [UIColor whiteColor];
    return loadingView;
}

// 创建加载错误视图
- (UIView *)createErrorView {
    UIView *errorView = [[UIView alloc] init];
    errorView.backgroundColor = [UIColor whiteColor];
    return errorView;
}

// 设置业务代码编译成的framework名字
- (void)fetchContextCodeWithPageName:(NSString *)pageName resultCallback:
    (KuiklyContextCodeCallback)callback {
    if (callback) {
        // 返回对应framework名字
        callback(@"shared", nil);
    }
}

- (void)dealloc {
    [[NSNotificationCenter defaultCenter] removeObserver:self];
}

// 指定当前页面获取 safeAreaInsets 的参考窗口
// 适用场景：当业务存在多 Window、悬浮窗、分屏等复杂场景，且框架默认获取的
safeAreaInsets 不正确时，
// 业务可以通过重写此方法，返回当前 Kuikly 页面所在的 window，以确保获取正确的安全区
域
- (UIWindow *)viewControllerHostWindow {
    return self.view.window;
}

```

@end

重要提示

必须使用 **KuiklyRenderViewControllerDelegator** 类，
不要使用 **KuiklyRenderViewControllerBaseDelegator**，否则会导致功能异常。

以 View 方式接入

Kuikly iOS Render 也支持以 view 承载 Page 将页面嵌入到任意原生视图层级中。可充分支持如瀑布流卡片、Banner 等卡片式结构场景。

接入步骤：在原生页面中创建 **KuiklyBaseView** 实例，指定 frame、页面名和页面参数，并触发生命周期方法即可。

反馈
文档
建议

代码示例

```
// 在 VC 的 viewDidLoad 中创建 KuiklyBaseView 并嵌入到自己的视图层级
@interface YourViewController ()
@property (nonatomic, strong) KuiklyBaseView *kuiklyView;
@end

- (void)viewDidLoad {
    [super viewDidLoad];

    NSDictionary *pageData = @{@"key": @"value"};

    // 创建 KuiklyBaseView, 指定 frame 和页面信息
    self.kuiklyView = [[KuiklyBaseView alloc] initWithFrame:CGRectMake(0, 0,
200, 300)

    pageName:@"YourPageName"

    pageData:pageData
    delegate:self
objc
```

```
frameworkName:@"shared"];
[self.view addSubview:self.kuiklyView];

// 手动触发生命周期
[self.kuiklyView viewWillAppear];
[self.kuiklyView viewDidAppear];
}

// 在 View 从屏幕移除时，手动触发 disappear
- (void)viewDidDisappear:(BOOL)animated {
    [super viewDidDisappear:animated];
    [self.kuiklyView viewDidDisappear];
}
```

完整的 View 粒度接入实践示例（卡片式风格瀑布流页面），请参考源码工程 iosApp 模块的 NativeAppWaterfallViewController 类，该 Demo 在一个 UICollectionView 的每个 Cell 中各嵌入了一个 KuiklyBaseView。

反馈文档建议

ViewController 接入方式的主要不同点

	VC 方式	View 方式
容器	KuiklyRenderViewController	KuiklyBaseView
尺寸控制	自动撑满整个 VC 的 view	通过 initWithFrame: 的 frame 参数自行指定尺寸
生命周期	VC 自动转发 viewWillAppear 等生命周期	需要业务手动调用 viewWillAppear 、 viewDidAppear 、 viewDidDisappear 等方法

	VC 方式	View 方式
初始化时机	VC 的 <code>viewDidLoad</code> 中由 <code>Delegator</code> 触发	<code>init</code> 时就执行 <code>loadKuiklyPage</code> ， 必须在 <code>viewDidLoad</code> 或之后创建
代理协议	实现 <code>KuiklyRenderViewControllerBaseDelegatorDelegate</code>	实现 <code>KuiklyViewBaseDelegate</code> （继承自同一个 <code>BaseDelegator</code> 协议，能力一致）

注意事项

- frame 不能为 zero：** `KuiklyBaseView` 初始化时 `frame.size` 不能为 `CGSizeZero`，建议传入最终显示尺寸以避免二次 layout
- 创建时机：** 由于 `KuiklyBaseView` 在 `init` 时就会执行页面加载，请确保在 VC 的 `viewDidLoad` 或之后创建，不要在 VC 的 `init` 阶段过早创建
- 生命周期手动转发：** 当 View 出现/消失时，需手动调用 `viewWillAppear`、`viewDidAppear`、`viewWillDisappear` 等方法
- 代理能力一致：** `KuiklyViewBaseDelegate` 继承自 `KuiklyRenderViewControllerBaseDelegatorDelegate`，与 VC 方式需要实现的代理方法完全一致（如 `fetchContextCodeWithPageName:`、`createLoadingView` 等）

注意

view 粒度接入进的原生页面，还要在路由处理类 `KRRouterHandler.m` 中,为新建的 `Native ViewController` 添加 `pageName` 路由分支。

实现Kuikly适配器（必须实现部分）

`Kuikly` 框架为了灵活和可拓展性，通过适配器的设计模式，将一些功能的具体实现委托给宿主 App 实现。

`Kuikly` 为 iOS 宿主工程提供了以下适配器

1. **图片加载适配器**: 用于给Kuikly的Image组件实现图片下载解码能力。**宿主侧必须实现**
2. **页面路由适配器**: 用于实现跳转到 Kuikly 容器。**宿主侧必须实现**
3. **日志适配器**: 用于给Kuikly框架和Kuikly业务实现日志打印。**推荐宿主侧实现**
4. **异常适配器**: 当Kuikly业务执行逻辑出错时，决定如何处理异常。**推荐宿主侧实现**
5. **颜色值转换适配器**: Kuikly框架对颜色值的处理，默认只处理十六进制的颜色值。**宿主按需实现**
6. **APNG图片加载适配器**: 用于给Kuikly提供APNG图片加载的能力。**宿主按需实现（使用APNG组件时必须实现）**
7. **PAG加载适配器**: 用于给Kuikly提供PAG动画播放能力。**宿主按需实现（使用PAG组件时需实现，如已依赖 `pod 'libpag'`，则无需额外实现）**

实现图片加载适配器

具体实现代码，请参考源码工程iOSApp模块的 `KuiklyRenderComponentExpandHandler` 类。

`KuiklyRenderComponentExpandHandler` 提供了以下三种图片加载方法：

推荐

-  `hr_setImageWithUrl:imageParams:complete:` — 完整支持（src一致性验证、错误回调、`imageParams`）
-  **废弃**（存在图片错乱风险，请勿使用）
- `hr_setImageWithUrl:forImageView:complete:` — 缺少 src 一致性验证、`imageParams`
- `hr_setImageWithUrl:forImageView:` — 仅支持基础加载

废弃原因说明

- `src`一致性验证 是保证图片准确加载的必要能力。后两种方法因不具备此能力，在页面多图片、图片src变更频繁 场景下易发生 图片错乱 现象，因此已废弃。
- `imageParams` 是src的补充参数，具体作用可见 [imageParams](#)

下面给出推荐方法的具体使用示例：

```
// .h
#import <Foundation/Foundation.h>
#import <OpenKuiklyIOSRender/KuiklyRenderBridge.h>

NS_ASSUME_NONNULL_BEGIN
```

objective-c

```

/*
 * @brief 需要接入方自定义实现图片加载
 */
@interface KuiklyRenderComponentExpandHandler :
NSObject<KuiklyRenderComponentExpandProtocol>

NS_ASSUME_NONNULL_END

@end

```

objective-c

反馈文档建议

```

// .m
#import "KuiklyRenderComponentExpandHandler.h"
#import <SDWebImage/UIImageView+WebCache.h>

@implementation KuiklyRenderComponentExpandHandler

+ (void)load {
    // 注册自定义实现
    [KuiklyRenderBridge registerComponentExpandHandler:[self new]];
}

/*
 * 自定义实现设置图片（带完成回调和src一致性验证）
 * @param loadURL 待加载图片的 url
 * @param imageParams 自定义的图片参数
 * @param complete 图片处理完成后的回调，内置src一致性验证
 * @return 是否处理该图片设置，返回值为YES，则交给该代理实现，否则sdk内部自己处理
 */
- (BOOL)hr_setImageWithUrl:(nonnull NSString *)loadURL imageParams:(NSDictionary*
_Nullable)imageParams complete:(ImageCompletionBlock)completeBlock;{
    // @warning 若使用SDWebImage加载图片，options 需设置为
    SDWebImageAvoidAutoSetImage = 1 << 10，避免自动设置图片导致图片错乱问题

    // 使用SDWebImage 实现图片加载
    [[SDWebImageManager sharedManager] loadImageWithURL:[NSURL
WithURLString:loadURL]

options:SDWebImageAvoidAutoSetImage

progress:nil
completed:^(UIImage * _Nullable
image, NSData * _Nullable data, NSError * _Nullable error, SDImageCacheType
cacheType, BOOL finished, NSURL * _Nullable imageURL) {

```

```

        // @warning 实现此方法时，必须在图片加载完成后调用completeBlock，SDK通过此回调完成UIImageView.image的最终设置，若不调用将导致图片无法显示
        if (completeBlock) {
            // * @warning 注意回调时所传入的imageURL是 hr_setImageWithURL的参数url，而非SDWebImage Block中返回的imageURL参数
            completeBlock(image, error, [NSURL URLWithString:loadURL]);
        }
    }];
    return YES;
}

...

@end

```

使用YYImage加载图片可参考：

objective-c

```

// 使用YYImage 实现图片加载
[[YYWebImageManager sharedManager] requestImageWithURL:[NSURL
    URLWithString:loadURL]
    options:YYWebImageOptionAvoidSetImage
    progress:nil
    transform:nil
    completion:^(UIImage * _Nullable
image, NSURL * _Nonnull url, YYWebImageFromType from, YYWebImageStage stage,
NSError * _Nullable error) {
    // @warning 实现此方法时，必须在图片加载完成后调用completeBlock，SDK通过此回调完成UIImageView.image的最终设置，若不调用将导致图片无法显示
    if (completeBlock) {
        // * @warning 注意回调时所传入的imageURL是 hr_setImageWithURL的参数url，而非SDWebImage Block中返回的imageURL参数
        completeBlock(image, error, [NSURL URLWithString:loadURL]);
    }
}];

```

反馈文档建议

完成后，可通过模版工程中的 ImageAdapter基准测试 页面来验证功能正常。

实现页面路由适配器

具体实现代码，请参考源码工程iOSApp模块的 KRRouterHandler 类。

objc

```
#import <Foundation/Foundation.h>
#import <OpenKuiklyIOSRender/KRRouterModule.h>
NS_ASSUME_NONNULL_BEGIN

@interface KRRouterHandler : NSObject<KRRouterProtocol>

@end

NS_ASSUME_NONNULL_END
```

objc

```
#import "KRRouterHandler.h"
#import "KuiklyRenderViewController.h"

@implementation KRRouterHandler

// 注册适配器
+ (void)load {
    [KRRouterModule registerRouterHandler:[self new]];
}

// 打开页面
- (void)openPageWithName:(NSString *)pageName pageData:(NSDictionary *)pageData
controller:(UIViewController *)controller {
    KuiklyRenderViewController *renderViewController =
[[KuiklyRenderViewController alloc] initWithPageName:pageName pageData:pageData];
    [controller.navigationController pushViewController:renderViewController
    animated:YES];
}

// 关闭页面
- (void)closePage:(UIViewController *)controller {
    [controller.navigationController popViewControllerAnimated:YES];
}

@end
```

反馈
文档
建议

实现日志适配器

该适配器Kuikly有默认实现，非必须实现，业务可根据实际使用需求来决定是否实现。具体实现代码，请参考源码工程core-render-ios模块的 KuiklyLogHandler 类。

objc

```

@interface KuiklyLogHandler : NSObject<KuiklyLogProtocol>

@end

@implementation KuiklyLogHandler

- (BOOL)asyncLogEnable {
    return NO;
}

- (void)logInfo:(NSString *)message {
    NSLog(@"%@", message);
}

- (void)logDebug:(NSString *)message {
    #if DEBUG
        NSLog(@"%@", message);
    #endif
}

    (void)logError:(NSString *)message {
        NSLog(@"%@", message);
    }

@end

```

反馈
文档
建议

实现异常适配器

该适配器非必须实现, 业务可根据实际使用需求来决定是否实现。如需实现, 需重写 `KuiklyRenderViewControllerBaseDelegatorDelegate` 协议的 `onUnhandledException:stack:mode:` 方法, 具体实现代码, 请参考源码工程 `iOSApp` 模块的 `KuiklyRenderViewController` 类。

objc

```

@implementation KuiklyRenderViewController

- (void)onUnhandledException:(NSString *)exReason stack:(NSString *)callstackStr
mode:(KuiklyContextMode)mode
{

```

```
// 处理异常或异常上报
}

@end
```

其他按需实现适配器示例参考[实现适配器（按需实现部分）](#)

链接Kuikly业务代码

Kuikly业务代码，在iOS平台上会被编译成 .framework 产物，下一步，我们将编译好的Kuikly业务代码 .framework 链接到你的工程。 .framework 的集成，你可以选择远程pod接入或者本地pod接入。这里为了方便，我只演示本地pod接入。

我们先前在 Kuikly [KMP跨端工程接入](#) 中已经新建了 Kuikly 业务工程，然后我们将这个业务工程的业务代码编译成的 .framework 链接到我们的现有iOS工程

```
source 'https://cdn.cocoapods.org/'

platform :ios, '14.1'

target 'KuiklyTest' do

  inhibit_all_warnings!

  pod 'OpenKuiklyIOSRender', 'KUIKLY_RENDER_VERSION'
  pod 'shared', :path => '/Users/XXX/workspace/TestKuikly/shared' # 本地存放Kuikly业务代码工程路径
end
```

重新执行 `pod install` 安装依赖

编写TestPage验证

1. 完成上述步骤后, 我们便完成了Kuikly的接入。下面我们在shared模块中编写TestPage, 验证是否接入成功。我们在之前[KMP跨端工程接入](#)中新建的工程中的 shared 模块中新建TestPage , 编写业务代码。新增的页面的名字为test

kt

```

@Page("test")
class TestPage : Pager(){
    override fun body(): ViewBuilder {
        return {
            attr {
                allCenter()
            }

            Text {
                attr {
                    fontSize(18f)
                    text("Hello Kuikly")
                    color(Color.GREEN)
                }
            }
        }
    }
}

```

反馈文档建议 接着我们初始化 KuiklyRenderViewController ，并将 test 作为 pageName 传入容器中, 指定跳转到我们刚新建的 TestPage 中

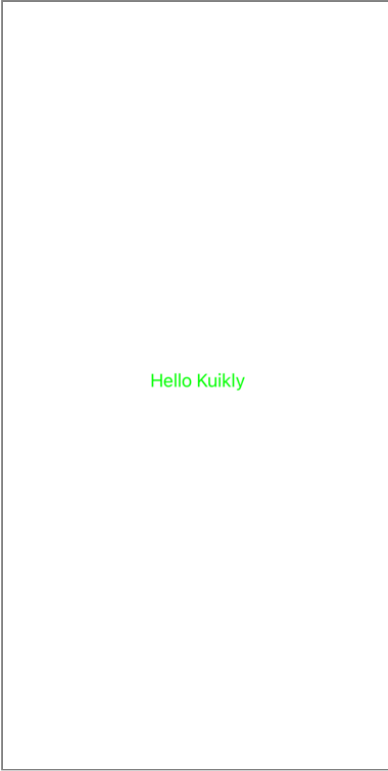
objc

```

- (void)pushKUIView {
    KuiklyRenderViewController *vc = [[KuiklyRenderViewController alloc]
initWithPageName:@"test" pageData:nil];
    [self.navigationController pushViewController:vc animated:YES];
}

```

3. 当手机出现以下界面时, 说明接入已经成功接入Kuikly



实现适配器（按需实现部分）

反馈
文档
建议

实现颜色值转换适配器

通过该适配器自定义颜色转换逻辑，业务可根据实际使用需求来决定是否实现 具体实现代码，请参考源码工程iOSApp模块的 `KuiklyRenderComponentExpandHandler` 类。

```
@implementation KuiklyRenderComponentExpandHandler
/*
 * 自定义实现设置颜色值
 * @param value 设置的颜色值
 * @return 完成自定义处理的颜色对象
 */
- (UIColor *)hr_colorWithValue:(NSString *)value {
    return nil;
}
```

objc

实现APNG图片加载适配器

通过该适配器加载并显示APNG图片，当业务需要使用APNG组件时必须实现该适配器 具体实现代码，请参考源码工程iOSApp模块的 `KRAPNGViewHandler` 类。

```

@interface KRAPNGViewHandler : SDAnimatedImageView<APNGImageViewProtocol>

@end

@implementation KRAPNGViewHandler

+ (void)load {
    [KRAPNGView registerAPNGViewCreator:^(id<APNGImageViewProtocol>
    _Nonnull)(CGRect frame) {
        KRAPNGViewHandler *apngView = [[KRAPNGViewHandler alloc]
initWithFrame:frame];
        return apngView;
    }]];
}

@end

```

objc

实现PAG适配器

反馈
文档
建议
PAGView 实例。业务可通过实现此适配器创建框架的 PAGView 组件，也可以构建自定义 PAGView，再通过 registerPAGViewCreator 输出实例。

业务可将PAGview实例与适配器单独的实现，也可以将PAGviewHandler适配器直接作为PAGview的实现。

代码示例

方式一 KRPAViewHandler 作为自定义PAGView实例且向外提供creator

```

// 适配器 自身继承 PAGView 并遵循协议（常见写法）
@interface KRPAViewHandler : PAGView <KRPAViewProtocol>
@end

@implementation KRPAViewHandler

+ (void)load {
    // 注册 Creator，返回遵循 KRPAViewProtocol 的实例
    [KRPAView registerPAGViewCreator:^(id<KRPAViewProtocol> _Nonnull(CGRect
frame) {
        return [[KRPAViewHandler alloc] initWithFrame:frame];
    }]];
}

```

objc

```

    }];
}

#pragma mark - KRPagViewProtocol

- (BOOL)setPath:(NSString *)filePath {
    // 加载 PAG 文件
}

- (void)play {
    // 开始播放动画
}

- (void)stop {
    // 停止播放动画
}

- (void)setProgress:(double)value {
    // 设置动画播放进度
}

// ...
@end

```

反馈
文档
建议

方式二 KRPAGViewHandler 仅作为工厂类

```

// 自定义 PAGView
@interface CustomPAGView : PAGView <KRPagViewProtocol>
@end

@implementation CustomPAGView
// 实现 KRPagViewProtocol 方法...
@end

// PAG适配器仅负责注册, 返回其他 View 实例
@implementation KRPAGViewHandler

+ (void)load {
    [KRPAGView registerPAGViewCreator:^(id<KRPagViewProtocol> _Nonnull(CGRect
frame) {

```

objc

```
        return [[CustomPAGView alloc] initWithFrame:frame]; // 返回其他实例
    }];
}

@end
```

注意

- 框架引入有libpag库，业务可直接继承libpag的PAGview，获取处理PAG的相关能力
- PAG适配器的定义与实现可参考类似的 `KRVideoViewHandler` 与 `KRAPNGViewHandler` 的实现